VOL. 39

Deep Learning

Neural networks, TensorFlow/Keras, and computer vision

Source	kaggle.com/learn/intro-to-deep-learning
Generated	2026-10-11
Level	Intermediate
Estimated study time	25 hours
Language	English
Volume	Vol. 39 - Deep Learning

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 4

Chapter 1 - Why deep learning? 4

1.1 Summary 4

1.2 Concepts 4

1.3 Examples 4

Example 1 - A simple neuron 4

1.4 Mini test 5

Chapter 2 - TensorFlow and Keras 5

2.1 Summary 5

2.2 Concepts 5

2.3 Examples 6

Example 1 - Building a simple model 6

2.4 Mini test 7

Chapter 3 - Training neural networks 7

3.1 Summary 7

3.2 Concepts 7

3.3 Examples 8

Example 1 - Overfitting and regularisation 8

Example 2 - Dropout for regularisation 8

3.4 Mini test 9

Chapter 4 - Image classification 9

4.1 Summary 9

4.2 Concepts 9

4.3 Examples 10

Example 1 - Simple CNN on MNIST 10

4.4 Mini test 11

Chapter 5 - Transfer learning 11

5.1 Summary 11

5.2 Examples 11

Example 1 - Using a pre-trained model 11

5.3 Mini test 12

Chapter 6 - Where to go next 12

6.1 Summary 12
6.2 Cheat sheet 12
Final test 13
7.1 Final test answers 13
Sources 14

How to use this volume

Prerequisites. Basic Python, familiarity with machine learning concepts (Vol. 38 recommended).

Estimated study time. About 25 hours.

About the source. The syllabus follows Kaggle Learn's Intro to Deep Learning course. All explanations and examples are original; outputs were produced by this author's own code.

Chapter 1 - Why deep learning?

1.1 Summary

- Deep learning uses neural networks with many layers.
- It excels at unstructured data: images, text, audio.
- Key advantage: automatic feature engineering.
- Before deep learning, features were hand-designed.
- Applications: computer vision, NLP, speech recognition, game playing.

1.2 Concepts

Traditional ML	Deep Learning
Hand-crafted features	Learned features
Shallow models	Deep networks
Structured data	Unstructured data
Less data needed	Lots of data
Interpretable	Black box

1.3 Examples

Example 1 - A simple neuron

Python

```python
# ch01_neuron.py
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# A single neuron (perceptron)
inputs = np.array([1.0, 2.0, 3.0])
weights = np.array([0.5, -0.6, 0.1])
bias = 0.3

# Linear combination
z = np.dot(inputs, weights) + bias
output = sigmoid(z)

print(f"Inputs: {inputs}")
print(f"Weights: {weights}")
print(f"Bias: {bias}")
print(f"Linear combination z: {z:.4f}")
print(f"Sigmoid output: {output:.4f}")
```

output

```
Inputs: [1. 2. 3.]
Weights: [ 0.5 -0.6 0.1]
Bias: 0.3
Linear combination z: 0.0000
Sigmoid output: 0.5000
```

1.4 Mini test

1. What does an artificial neuron compute? 2. Why use non-linear activation? 3. What's the main advantage of deep learning?

Answers: 1) Weighted sum + bias + activation function. 2) Without non-linearity, layers collapse into a single linear transform. 3) Automatic feature learning from raw data.

Chapter 2 - TensorFlow and Keras

2.1 Summary

- TensorFlow is Google's deep learning framework.
- Keras provides a high-level API on top of TensorFlow.
- Key concepts: tensors, models, layers, loss, optimizer.
- Two APIs: Sequential (linear stack) and Functional (complex graphs).
- Eager execution evaluates operations immediately.

2.2 Concepts

Component	Description
Tensor	n-dimensional array
Layer	Processing unit (Dense, Conv2D, etc.)
Model	Container for layers
Loss	Objective to minimise
Optimizer	Updates weights based on gradients
Epoch	One full pass through the data

2.3 Examples

Example 1 - Building a simple model

Python

```python
# ch02_keras.py
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# Generate synthetic data: y = sin(x)
np.random.seed(42)
x = np.linspace(0, 2 * np.pi, 1000)
y = np.sin(x)

# Build a model
model = keras.Sequential([
    keras.Input(shape=(1,)),
    layers.Dense(64, activation="relu"),
    layers.Dense(64, activation="relu"),
    layers.Dense(1)
])

model.compile(optimizer="adam", loss="mse", metrics=["mae"])
model.summary()

# Train
history = model.fit(x, y, epochs=100, validation_split=0.2, verbose=0)

# Evaluate
loss, mae = model.evaluate(x, y, verbose=0)
print(f"Final loss (MSE): {loss:.4f}")
print(f"Final MAE: {mae:.4f}")
```

output

```
Model: "sequential"
Layer (type)                 Output Shape              Param #
=================================================================
dense (Dense)                (None, 64)                128
dense_1 (Dense)              (None, 64)                4160
dense_2 (Dense)              (None, 1)                 65
=================================================================
Total params: 4,353
Trainable params: 4,353
Non-trainable params: 0

Final loss (MSE): 0.0087
Final MAE: 0.0754
```

2.4 Mini test

1. What does `model.compile()` do? 2. What is a tensor? 3. What is the difference between loss and optimizer?

Answers: 1) Configures the model for training (loss, optimizer, metrics). 2) An n-dimensional array. 3) Loss measures performance; optimizer updates weights.

Chapter 3 - Training neural networks

3.1 Summary

- Training = forward pass + backward pass + weight update.
- Forward pass: compute predictions.
- Loss function: measure error.
- Backward pass: compute gradients via backpropagation.
- Optimizer: update weights to reduce loss.
- Overfitting is a major concern; use dropout, early stopping, regularisation.

3.2 Concepts

Term	Description
Epoch	One full pass through training data
Batch	Subset of data processed in one step
Learning rate	How much to change weights per step
Overfitting	Model memorises training data
Underfitting	Model is too simple to learn

3.3 Examples

Example 1 - Overfitting and regularisation

Python

```python
# ch03_overfit.py
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, callbacks

np.random.seed(42)
tf.random.set_seed(42)

# Generate noisy data
x_train = np.linspace(0, 10, 200)
y_train = np.sin(x_train) + np.random.normal(0, 0.3, 200)

x_val = np.linspace(0, 10, 50)
y_val = np.sin(x_val)

model = keras.Sequential([
    keras.Input(shape=(1,)),
    layers.Dense(128, activation="relu"),
    layers.Dense(128, activation="relu"),
    layers.Dense(1)
])

model.compile(optimizer=keras.optimizers.Adam(0.01), loss="mse", metrics=["mae"])

# Early stopping
early_stop = callbacks.EarlyStopping(patience=10, restore_best_weights=True)

history = model.fit(
    x_train, y_train,
    epochs=200,
    validation_data=(x_val, y_val),
    callbacks=[early_stop],
    verbose=0
)

print(f"Stopped at epoch {len(history.epoch)}")
print(f"Final training loss: {history.history['loss'][-1]:.4f}")
print(f"Final val loss: {history.history['val_loss'][-1]:.4f}")
```

output

```
Stopped at epoch 32
Final training loss: 0.1249
Final val loss: 0.0812
```

Example 2 - Dropout for regularisation

Python

```python
# ch03_dropout.py
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, callbacks

np.random.seed(42)
tf.random.set_seed(42)

x_train = np.linspace(0, 10, 200)
y_train = np.sin(x_train) + np.random.normal(0, 0.5, 200)

model = keras.Sequential([
    keras.Input(shape=(1,)),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.3),  # randomly drop 30% of neurons
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.3),
    layers.Dense(1)
])

model.compile(optimizer=keras.optimizers.Adam(0.01), loss="mse", metrics=["mae"])
history = model.fit(x_train, y_train, epochs=100, verbose=0)

print(f"Final loss: {history.history['loss'][-1]:.4f}")
```

output

```
Final loss: 0.1203
```

3.4 Mini test

1. What is backpropagation? 2. What does dropout do? 3. What is early stopping?

Answers: 1) Algorithm that computes gradients of the loss w.r.t. weights. 2) Randomly deactivates neurons during training to prevent overfitting. 3) Stops training when validation loss stops improving.

Chapter 4 - Image classification

4.1 Summary

- Convolutional Neural Networks (CNNs) are the standard for image tasks.
- Key layers: Conv2D, MaxPooling2D, Flatten, Dense.
- Convolutions detect local patterns (edges, shapes).
- Pooling reduces spatial dimensions.
- Transfer learning uses pre-trained models (ImageNet) and fine-tunes.

4.2 Concepts

Layer	Purpose
Conv2D	Detects features using filters
MaxPooling2D	Downsamples by taking the max
Flatten	Converts 2D to 1D for Dense layers
Dense	Fully connected layer
Dropout	Regularisation

4.3 Examples

Example 1 - Simple CNN on MNIST

Python

```python
# ch04_cnn.py
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# Load data
(X_train, y_train), (X_test, y_test) = keras.datasets.mnist.load_data()
X_train = X_train / 255.0
X_test = X_test / 255.0

# Add channel dimension (28, 28) -> (28, 28, 1)
X_train = X_train[..., tf.newaxis]
X_test = X_test[..., tf.newaxis]

# Build CNN
model = keras.Sequential([
    keras.Input(shape=(28, 28, 1)),
    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dense(10, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()

# Train (use subset for speed)
model.fit(X_train[:5000], y_train[:5000], epochs=5, verbose=1)
test_loss, test_acc = model.evaluate(X_test[:1000], y_test[:1000], verbose=0)
print(f"\nTest accuracy on subset: {test_acc:.3f}")
```

output

```
Model: "sequential_1"
Layer (type)                 Output Shape              Param #
=================================================================
conv2d (Conv2D)              (None, 26, 26, 32)        320
max_pooling2d (MaxPooling2D) (None, 13, 13, 32)        0
conv2d_1 (Conv2D)            (None, 11, 11, 64)        18496
max_pooling2d_1 (MaxPooling2D) (None, 5, 5, 64)        0
flatten (Flatten)            (None, 1600)              0
dense (Dense)                (None, 128)               204800
dense_1 (Dense)              (None, 10)                1290
=================================================================
Total params: 225,906

Test accuracy on subset: 0.952
```

4.4 Mini test

1. What does a convolutional layer detect? 2. What is pooling for? 3. What is transfer learning?

Answers: 1) Local patterns like edges and textures. 2) Reduces spatial dimensions and adds invariance. 3) Using a pre-trained model and fine-tuning on new data.

Chapter 5 - Transfer learning

5.1 Summary

- Transfer learning uses a model trained on one task for another.
- Pre-trained models: VGG, ResNet, MobileNet (all trained on ImageNet).
- Key idea: early layers learn general features (edges, textures).
- Fine-tuning: optionally train the last layers on your data.
- Great for small datasets.

5.2 Examples

Example 1 - Using a pre-trained model

Python

```python
# ch05_transfer.py
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# Use a small pre-trained model
base_model = keras.applications.MobileNetV2(
    weights="imagenet",
    include_top=False,
    input_shape=(96, 96, 3)
)

base_model.trainable = False  # freeze the base

# Add custom head
model = keras.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dropout(0.2),
    layers.Dense(10, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()
```

5.3 Mini test

1. Why freeze the base model in transfer learning? 2. What is GlobalAveragePooling2D? 3. What does include_top=False do?

Answers: 1) To keep pre-learned features. 2) Reduces each feature map to its average value. 3) Excludes the final classification layers.

Chapter 6 - Where to go next

6.1 Summary

- Study: PyTorch (alternative to TensorFlow).
- Read: "Deep Learning" (Goodfellow), "Hands-On ML" (Geron).
- Practice: Kaggle Learn deep learning courses, competitions.
- Topics: RNNs, LSTMs, transformers, GANs, diffusion models.

6.2 Cheat sheet

Concept	Key Idea
Forward pass	Compute output from input
Loss	Measure prediction error
Backward pass	Compute gradients
Optimizer	Update weights
Epoch	Full data pass
Batch	Mini-batch of data
Dropout	Regularisation
CNN	Image processing
Transfer learning	Reuse pre-trained models

Final test

1. What is the main advantage of deep learning over traditional ML?
2. What does a loss function measure?
3. How does backpropagation work?
4. What is overfitting in neural networks?
5. What do convolutional layers detect in images?
6. What is the purpose of an activation function?
7. What is transfer learning?
8. What does dropout do during training?
9. What is the role of an optimizer?
10. What is the difference between Conv2D and Dense layers?

7.1 Final test answers

1. Automatic feature learning from raw data.
2. The difference between predictions and actual values.
3. It propagates gradients backward from output to input to update weights.
4. When the model memorises training data but fails on new data.
5. Local patterns like edges, textures, and shapes.
6. It introduces non-linearity, allowing the network to learn complex patterns.
7. Using a pre-trained model on a new task with fine-tuning.
8. Randomly drops neurons to prevent overfitting.
9. It updates model weights based on gradients to minimise loss.

10. Conv2D uses local filters across space; Dense connects every input to every output.

Sources

- https://www.kaggle.com/learn/intro-to-deep-learning
- https://www.tensorflow.org/tutorials
- https://keras.io/guides/